

전성훈

iOS Developer

✉ jsh097610@gmail.com | github.com/Jeon0976 | velog.io/@jeon0976
in [linkedin.com/in/seonghun-jeon](https://www.linkedin.com/in/seonghun-jeon) | jeon0976.github.io/portfolio-site

SUMMARY

명확하게 책임을 나눠 유지보수와 확장이 쉬운 코드를 작성하는 데 중점을 둡니다. 기술은 팀 상황과 트레이드오프를 따져 고릅니다. 한 제품의 iOS를 설계부터 출시와 운영까지 단독으로 맡았고, 같은 제품의 Android 출시와 WKWebView 전환까지 이어서 담당했습니다.

EXPERIENCE

iOS 개발자 | 키라&컴퍼니

2025.05 – 2026.07 (1년 3개월)

금융 교육 게이미피케이션 서비스 KIRA(Katsby) · 초기 스타트업 멤버

iOS 단독 주도, Android 출시, 웹 전환

사용기술 SwiftUI(iOS 17+), Clean Architecture + MVI, Tuist, Swift Concurrency, Combine, WebKit, URLSession, StoreKit 2, Swinject, Firebase, Amplitude, AppsFlyer

- **변화 데이터와 애니메이션 상태를 각각 다른 계층에서 관리**, 퀘스트 보상은 이전 상태 보관과 전후 비교를 actor 저장소가 맡고 애니메이션 매니저는 계산된 변화만 받아 순서와 표현만 담당
 - 변화를 값으로 모델링해 한 화면에서 놓쳐도 다른 화면이 이어서 표시
 - 단계 타입으로 애니메이션 순서를 강제하고 화면 이탈 시 취소
 - 마지막 본 주기를 영속 저장해 겹다 쳐도 재실행 안 됨
- **불가능한 상태를 타입에서 제거**, 퀴즈 진행처럼 갈래가 많은 흐름은 Store가 상태를 타입으로 들고 다음 단계 판정은 레슨 상태만 받는 순수 함수 UseCase로 분리. 기능이 늘어나도 검증할 조합이 곱으로 늘지 않음
- **화면 전환과 서버 반영의 비동기 경계 설계**, 응답을 기다려야 다음 문항으로 넘어가던 구조에서 둘을 분리하고 진행 중인 제출 수를 세어 체크포인트에서 0이 될 때까지 대기, 실패분은 큐에 모아 경계에서 재전송
- **탭 전환 애니메이션을 상태 단위로 설계**, 형태가 다른 두 종류의 탭바를 오가야 해서 너비, 높이, 아이콘의 투명도와 크기를 상태 하나에 묶고 스프링 하나로 제어. 값을 하나씩 잡아 가며 탭이 바뀌는 동안 바가 끊기지 않고 이어지도록 맞춤
- **탭 전환 severe hang 해소**, Instruments로 측정해 성격이 다른 상태들이 ObservableObject 하나에 모여 값 하나만 바뀌어도 구독 뷰가 전부 재렌더되는 것을 원인으로 확인, 상태 수명 범위 기준으로 @Observable Store들로 분리하고 각 뷰가 실제로 읽는 것만 참조하도록 정리
- **앱 런치 간헐 크래시를 DI 컨테이너의 스레드 안전성 문제로 진단**, Crashlytics 스택 트레이스를 따라가 Swinject Container.resolve()가 스레드 세이프하지 않은 것을 확인하고 synchronize() 기반 resolver로 전환
- **URLSession 기반 커스텀 APIClient 전면 재설계**, Moya의 콜백 경계에서 Task 취소가 전파되지 않아 화면 이탈 후에도 요청이 살아 있던 문제를 해결. session-plugins-retryPolicy 주입, isIdempotent 비명등 가드, 계층형 에러
- 타입이 지워진 NavigationPath 대신 [Route] 배열을 Router가 직접 소유해 중복 push 차단과 특정 화면 복귀 구현, 탭별 스택 보존 탭바·전역 오버레이·바텀시트 코디네이터까지 화면 인프라 직접 설계
- **딥링크 보류와 재개**, 푸시·URL·디퍼드 딥링크(AppsFlyer OneLink)를 파서와 라우터로 나누고 온보딩이나 오버레이 위로 도착하면 보류했다 닫힌 뒤 최신 한 건만 1회 실행
- **WKWebView 전환에서 네이티브와 웹이 지킬 규칙을 정하고 양쪽에 구현**, 세션이 바뀌면 웹 캐시를 비우고, 잘못 올라온 Safe Area 값은 적용하지 않으며, 배경 스크롤 잠금은 웹 CSS로는 뚫려 WKWebView 터치까지 막음. 브리지는 목으로 갈아 끼울 수 있게 만들어 앱 없이도 웹만으로 확인 가능
- StoreKit 2 인앱 결제를 actor 리포지토리와 Transaction.updates로 구성, 미완료 거래를 앱 재실행 때 자동 복구하고 서버 지급을 확인한 뒤에 거래를 마침
- **Android 개발자와 함께 앱을 구현해 정해진 출시 일정을 지킴**, 판정 규칙과 화면 상태, 예외를 iOS에서 먼저 명세로 정리해 덕분에 새로 설계하지 않고 옮기는 작업이 됨. 두 플랫폼이 같은 규칙 위에서 동작하도록 맞춤
- WebView 전환의 React/Next.js 화면은 요구사항과 구조를 직접 잡고 Claude Code로 구현해 배포, 생성 코드는 실제 Safari와 WKWebView에서 확인하고 고쳐 통합
- **AI 브리핑 콘텐츠 파이프라인을 설계**해 생성부터 품질 검사, 발행까지 사람 개입 없이 매일 자동으로 돌게 함

- 보상 축소가 결정된 상황에서 어디까지 내려도 이탈이 없는지를 A/B 테스트로 설계해 검증, 사용자를 활동 빈도로 나눠 구간별 이탈을 비교하고 보상 정책을 확정
- 개발 외 기획 회의, 채용, 온보딩, CS까지 참여하며 초기 스타트업의 제품 결정에 함께 관여

iOS 개발자 | 일루미나리안

2024.07 – 2025.01 (7개월)

클라우드 펀딩 앱 MIRAOTO · 신규 프로젝트 아키텍처 설계·기능 개발

사용기술 Swift, MVVM + Combine(단방향), Coordinator + DIContainer, Moya

- 신규 프로젝트의 내비게이션 아키텍처를 제안해 확정하고 구현까지 진행
- 신규 프로젝트의 3D 카드 표현을 검증하는 SceneKit 데모 구현, Blender 모델을 노드 계층(카메라·조명·재질)으로 통합하고 SCNAAction 인터랙션 적용 (최소 예제로 다시 만들어 블로그에 정리)

모바일(iOS/Android) 개발자 | 우리요

2024.01 – 2024.04 (4개월)

HR·근태관리 서비스(알파요·핀플) · 운영·기능 개발

사용기술 Swift, UIKit, AutoLayout, Instruments

- Instruments로 메모리 누수 해결, 클로저 순환 참조를 찾아 화면 전환 구조 정리
- MVC 패턴을 MVVM으로 개선해 조금이나마 고치기 쉬운 코드로 전환
- StoryBoard를 코드 기반 오토레이아웃으로 전환
- iOS 개발과 함께 Android QA 대응까지 맡음

UAV 조종사·선임 교관 | 대한민국 육군 (중사 전역)

2015.08 – 2022.08 (7년)

- 대대급 무인항공기(UAV) 조종사·선임 교관, 5년간 500회 이상 무사고 비행. 부대에 처음 생긴 신규 보직이라 참고 절차는 있었으나 운용 세부가 부족해 매뉴얼을 보완하며 절차를 세움
- 비상상황 조치를 실경험 기반으로 문서화해 사단과 군단에 전파, 기체 손상 확률을 낮춰 정비 유공 표창
- 민가 화재 초동조치로 소방 표창 ([철원신문 보도](#))

PROJECTS

iOS-TheModularArchitecture-Sample

2026

같은 앱을 Clean ↔ Micro Feature 두 구조로 구현해 비교 · github.com/Jeon0976/iOS-TheModularArchitecture-Sample

사용기술 Swift 6, Tuist, Micro Feature Architecture, Swift Concurrency, XCTest

- 레이어 대신 피쳐로 나누고 피쳐당 Interface / Sources / Testing / Tests / Demo(Example) 타겟을 분리해 계약만 노출하는 경계를 설계
- 이미지 파이프라인 책임 분리, 저장소는 Data만 반환하고 다운샘플링과 2계층 캐시(히트·미스 통계)는 화면 쪽에 배치. 저장소가 렌더링을 모르니 UIKit 의존 없이 단위 테스트 작성
- 화면을 닫으면 진행 중인 요청이 실제로 끊기고 늦게 도착한 응답이 최신 결과를 덮지 않도록 타입으로 보장, 호출부가 매번 챙기지 않아도 되게 함

AVIRO

2023.05 – 2026.04 (약 3년)

전국 비건 식당 공유 플랫폼 · 동아리 팀 중 iOS 전담 · App Store 출시·운영(종료) · 누적 1,300명+ / 평점 4.9 · github.com/AVIRO-official/AVIRO-iOS

사용기술 Swift, UIKit(코드 기반), MVP, Realm

- 전부 코드 기반으로 UICollectionView/UITableView 커스텀
- MVP 패턴으로 최초 서비스를 구현하고 운영
- 중간에 합류한 Android 팀원 온보딩 가이드 작성

SERENADE

2025.02 – 2025.07 (6개월)

웨딩 정보 서비스 · iOS 2인 · 체크리스트, 예산, 마이페이지 담당 · App Store

사용기술 Swift, ReactorKit(단방향), RxSwift, RxCocoa, RxDataSources, Tuist, Kingfisher

- ReactorKit 단방향 상태 관리, RxDataSources 섹션 diff, 체크 토글은 낙관적 업데이트로 UI 선반영 후 서버 반영

SpeechBubble

커스텀 말풍선 라이브러리 · github.com/Jeon0976/SpeechBubble

사용기술 Swift, UIKit, UIBezierPath, OptionSet API, SPM, CocoaPods 배포

TECHNICAL SKILLS

언어	Swift, Objective-C(유지보수)
UI	SwiftUI, UIKit
아키텍처	Clean Architecture, Micro Feature Architecture, MVI, MVVM-C, Tuist
비동기	Swift Concurrency, Combine, RxSwift
네이티브·웹	WKWebView, React/Next.js
네트워크·저장소	URLSession, CoreData, Keychain
테스트·도구	XCTest, Instruments, SPM, Firebase, Amplitude
AI	Claude Code, Codex

EDUCATION

서울사이버대학교

2019.03 – 2023.02

소프트웨어학과 학사

OTHERS

오픈소스 컨트리뷰터 | **iOS-Clean-Architecture-MVVM (kudoleh)**

github.com/kudoleh/iOS-Clean-Architecture-MVVM

CERTIFICATIONS

정보처리기사	2022.11
SQLD	2023.04
리눅스마스터 2급	2023.03
네트워크관리사 2급 필기	2024.02

LANGUAGE

영어 (TOEIC 810) 2023.02

WORK IN DETAIL

복잡한 상태 설계

캐치비는 베타 출시를 일주일 앞두고 합류해 서비스가 끝날 때까지 iOS를 맡은 제품입니다. 금융 교육을 게임처럼 만든 앱이라 화면마다 상태가 많았고 매주 기능이 새로 붙었습니다. 1년 3개월 동안 화면 구현부터 네트워크, 내비게이션, 결제, 분석까지 iOS 전반을 혼자 결정하고 책임졌습니다. 처음부터 끝까지 제 손으로 끌고 간 제품이라 애착이 큼니다.

대표적인 기능이 퀘스트 보상입니다. 퀘스트를 달성하면 진행 바가 차오르고 보상이 날아가고 완료 표시가 뜨는데, 이 셋이 순서대로 한 번씩만 나와야 합니다. 어려운 지점은 무엇이 바뀌었는지를 어디서 판단하느냐였습니다. 화면이 판단하면 그 화면에 들어와 있어야만 알 수 있어서, 퀴즈를 풀다 보상을 받고 빠져나가면 이미 바뀐 값은 아무에게도 보이지 않은 채 사라집니다.

그래서 변화를 판단하는 일을 데이터 영역으로 옮겼습니다. 저장소가 마지막으로 사용자에게 보여준 스냅샷과 지금 상태를 함께 들고 둘을 비교해 아직 안 보여준 변화만 값으로 만들어 넘깁니다. 화면은 그 값을 받아 순서대로 보여주지만 합니다. 여러 화면이 동시에 읽고 쓰기 때문에 actor로 직렬화했고 스냅샷은 애니메이션이 끝난 뒤에만 확정합니다. 그래서 퀴즈 화면에서 놓친 변화를 퀘스트 보드가 이어서 보여주고 앱을 껐다 켜도 남습니다.

보여주는 쪽은 따로 뒀습니다. 애니메이션 단계를 타입으로 정의해 순서를 강제하고 화면을 벗어나면 취소합니다. 무엇이 달라졌는지는 데이터 영역이 알고, 그걸 어떻게 보여줄지는 화면이 압니다.

나눠 두니 두 쪽이 서로를 몰라도 됐습니다. 나중에 재화를 얻는 보상 애니메이션이 중간에 하나 더 붙었을 때도 변화를 판단하는 코드는 그대로 두고 보여주는 순서만 고치면 됐습니다. 매주 기능이 붙는 일정에서 손댈 자리가 정해져 있으니 요구가 늘어도 그만큼 빠르게 붙일 수 있었습니다.

사용자 경험 개선

퀴즈는 캐츠비의 핵심 콘텐츠였습니다. 사용자가 앱에서 가장 오래 머무는 곳이라 여기서 생기는 지연은 한 번으로 끝나지 않고 문제를 풀 때마다 반복됩니다.

문제를 풀면 서버 응답을 기다린 뒤에 다음으로 넘어가서, 맞힌 걸 알면서도 매번 한 박자씩 멈췄습니다. 화면 전환과 서버 반응을 떼어내 진행은 곧바로 이어지게 하고, 정합이 필요한 체크포인트에서만 진행 중인 제출이 끝나기를 기다리게 했습니다. 실패한 제출은 큐에 모아 그 지점에서 다시 보냅니다.

퀴즈맵은 그 콘텐츠로 들어가는 입구입니다. 챕터별 레슨을 세로로 길게 잇는 스크롤 화면이라, 한 코스가 끝나는 지점에 스크롤이 닿고 다음 코스가 있으면 그때 다음 코스를 미리 받아 둡니다. 코스의 마지막 레슨을 풀고 있을 때도 같은 준비를 합니다. 사용자가 다음 코스로 넘어가면 받아 둔 것을 그대로 써서 네트워크 없이 바로 그립니다.

미리 받아 두는 쪽만 만들면 오히려 어긋납니다. 스크롤이 오르내리며 여러 번 요청되지 않게 묶었고 코스가 바뀌면 받아 둔 내용과 트리거 조건을 함께 비웁니다. 화면을 벗어나면 진행 중인 요청을 취소합니다. 코스를 넘나들 때 조금이라도 자연스럽게 이어지도록 붙인 장치들입니다.

제가 만드는 서비스는 하루에도 수십 번씩 직접 눌러 보고 실제로도 씁니다. 그러면서 걸리는 지점과 비슷한 서비스를 보며 아쉬웠던 지점을 목록으로 모아 두고, 일정이 허락하는 만큼 하나씩 처리해 왔습니다. 앞의 두 가지도 그 목록에서 나왔습니다.

변경에 대응하는 구조

첫 앱 AVIRO는 데이터와 로직, 화면이 한 곳에 섞인 채로 출시했습니다. 운영 중 대규모 업데이트가 들어왔을 때 어디를 고치면 어디가 깨지는지 알 수 없어 손대지 못한 지점이 생겼습니다. 그때부터 구조를 먼저 잡는 쪽으로 방향을 바꿨고 다음 실무부터는 계층을 나눠 설계했습니다.

실무에서 반영한 건 캐츠비였습니다. 퀴즈는 처음에 학습과 복습 정도만 있어서 불리언 몇 개로 화면 상태를 표현했습니다. 설문과 결말, 완료가 차례로 붙으면서 조합이 곱으로 늘었고 복습이면서 완료 같은 있을 수 없는 상태가 컴파일을 통과했습니다. 진행 단계를 타입으로 바꾸니 단계가 하나 늘 때마다 컴파일러가 처리하지 않은 자리를 전부 짚어 줘서, 상태가 더 붙어도 빠뜨리는 곳 없이 확장할 수 있었습니다. 다음 단계를 정하는 규칙은 레슨 상태만 받는 순수 함수로 빼 화면 없이 확인할 수 있게 했습니다.

운영하면서 계층만으로는 아쉬운 지점도 봤습니다. 기능이 커질수록 하나를 고치는 데 열어야 할 모듈이 늘었고, 테스트를 잘게 나누기 어려웠습니다. 그 지점은 개인 프로젝트에서 시험했습니다. 같은 앱을 계층 단위와 피쳐 단위로 두 번 만들면서 피쳐마다 모듈을 나누고 계약만 밖으로 내보내, 화면 하나를 고칠 때 그 피쳐만 열면 되게 했습니다. 대표적으로 이미지 파이프라인은 저장소가 데이터만 돌려주고 크기를 줄이는 일과 캐시는 화면 쪽에 뒀습니다. 저장소가 그리는 일을 모르니 UI 프레임워크 없이 확인할 수 있습니다.

나중에 나누는 비용이 지금 나누는 비용보다 훨씬 크다는 걸 한 번 겪었습니다. 그래서 일정이 빠듯할 때도 어느 코드가 어디까지 책임지는지는 먼저 정해 두고 시작합니다.

AI로 만든 생산성

Android는 출시일이 정해져 있는데 인력이 부족했습니다. iOS를 만들면서 판정 규칙과 화면 상태, 예외 처리를 명세로 정리해 둔 것이 있어서, 그 명세를 기준으로 Android 개발자와 함께 Claude Code로 구현했습니다. 새로 설계할 것이 없었으니 두 플랫폼이 같은 규칙 위에서 도는지만 확인하면 됐고 정해진 일정을 지켜 출시했습니다.

WebView 전환의 React와 Next.js 화면도 같은 방식이었습니다. 익숙하지 않은 스택이라 요구사항과 구조를 먼저 잡고 구현을 맡겼습니다. 브라우저에서 되던 것이 웹뷰에서 안 되는 경우가 있어서, 실제 Safari와 WKWebView에서 하나씩 확인하고 고쳐 넣었습니다.

브리핑 콘텐츠 파이프라인은 사람이 매일 손대지 않아도 돌아가야 했습니다. 생성부터 품질 검사, 발행까지 자동으로 이어지게 설계하고 검사를 통과하지 못한 것은 나가지 않게 했습니다.

셋 다 구조와 판단은 제가 하고 구현 속도를 AI로 벌었습니다. 그렇게 아낀 시간은 생성된 것이 실제 환경에서 도는지 확인하는 데 썼습니다.